

1. 哈希

unordered_set

unordered_map

i. 插入和删除

- insert(value) - 插入元素
- erase(key) - 删除指定键的元素
- clear() - 清空哈希表

ii. 访问和查询

- find(key) - 查找键，返回迭代器，未找到返回 end()
- count(key) - 统计键出现次数 (0 或 1)
- size() - 返回哈希表大小
- empty() - 检查哈希表是否为空

2. 字符串

stringstream 把字符串改变为了流

```
stringstream ss(paragraph); //便于 识别 单词
```

i. 查找和截取

- substr(pos, len) - 从位置 pos 开始，截取 len 个字符（默认到末尾）
- find(str) - 返回子串首次出现的位置，未找到返回 string::npos
- find(str, pos) - 从位置 pos 开始查找子串
- rfind(str) - 从后向前查找子串

ii. 长度和容量

- length() 或 size() - 返回字符串长度
- empty() - 检查字符串是否为空
- capacity() - 返回分配的容量

iii. 添加和删除

- push_back(ch) - 末尾添加字符
- pop_back() - 删除末尾字符
- append(str) - 末尾追加字符串
- insert(pos, str) - 在位置 pos 插入字符串
- erase(pos, len) - 删除从 pos 开始的 len 个字符
- clear() - 清空字符串

iv. 比较和替换

- compare(str) - 字符串比较，返回 0/正数/负数
- replace(pos, len, str) - 替换从 pos 开始的 len 个字符为 str

v. 访问

- at(index) 或 [index] - 访问指定位置的字符（at 会检查越界）

vi. 转换函数

- `stoi(str)` - 字符串转整数
- `stof(str)` 或 `stod(str)` - 字符串转浮点数
- `to_string(num)` - 数字转字符串
- `tolower(ch)` - 字符转小写
- `toupper(ch)` - 字符转大写

vii. 增删改查

- `insert(pos, str)` - 在位置 `pos` 插入字符串
- `erase(pos, len)` - 删除从 `pos` 开始的 `len` 个字符
- `replace(pos, len, str)` - 替换从 `pos` 开始的 `len` 个字符为 `str`
- `find(str)` - 查找子串位置
- `substr(pos, len)` - 截取子串
- `pop_back()` - 删除末尾字符
- `push_back(ch)` - 末尾添加字符

viii. •

3. 数组

vector

i. 添加和删除

- `push_back(value)` - 在末尾添加元素
- `pop_back()` - 删除末尾元素
- `insert(pos, value)` - 在位置 `pos` 插入元素
- `erase(pos)` - 删除位置 `pos` 的元素
- `clear()` - 清空数组

ii. 访问和查询

- `at(index)` 或 `[index]` - 访问指定位置的元素 (`at` 会检查越界)
- `size()` - 返回数组大小
- `empty()` - 检查数组是否为空
- `front()` - 返回第一个元素
- `back()` - 返回最后一个元素

iii. 排序和查找

- `sort(vec.begin(), vec.end())` - 对数组进行排序
- `find(vec.begin(), vec.end(), value)` - 查找元素位置, 未找到返回 `vec.end()`

4. 常用计算技巧

- `accumulate(vec.begin(), vec.end(), 0)` - 求和 (需 `#include <numeric>`)
- `sort(vec.begin(), vec.end())` - 排序 (需 `#include <algorithm>`)
- `reverse(vec.begin(), vec.end())` - 反转

- `unique(vec.begin(), vec.end())` - 去重 (需配合排序和`erase`)
- `copy(vec.begin(), vec.end(), dest.begin())` - 拷贝
- `count(vec.begin(), vec.end(), value)` - 统计某值出现次数
- `max_element(vec.begin(), vec.end())` - 最大值
- `min_element(vec.begin(), vec.end())` - 最小值

示例：

```
int suma = accumulate(aliceSizes.begin(), aliceSizes.end(), 0); // 求和
sort(aliceSizes.begin(), aliceSizes.end()); // 排序
```

5. GCD

- `gcd(a, b)` - 计算 a 和 b 的最大公约数 (需 `#include <numeric>`)
- `lcm(a, b)` - 计算 a 和 b 的最小公倍数 (需 `#include <numeric>`)
- `__gcd(a, b)` - 计算 a 和 b 的最大公约数 (GCC 内置函数)
- `__lcm(a, b)` - 计算 a 和 b 的最小公倍数 (GCC 内置函数)